

ROS 操作系统入门

机器人开发的“通用语言”

为什么我们需要 ROS?

- **痛点:**

机器人开发涉及传感器驱动、运动控制、算法决策等，模块间通信极其复杂。

每次换个底盘都要重写底层驱动？

每次更改数据都要拆卸零件？

- **ROS 的价值:**

- 提供标准化的通信机制（解耦）。
- 拥有丰富的开源生态（导航、视觉等）。
- 强大的调试与仿真工具。



核心价值: 在于提供标准化通信机制和丰富工具生态，减少低效的重复

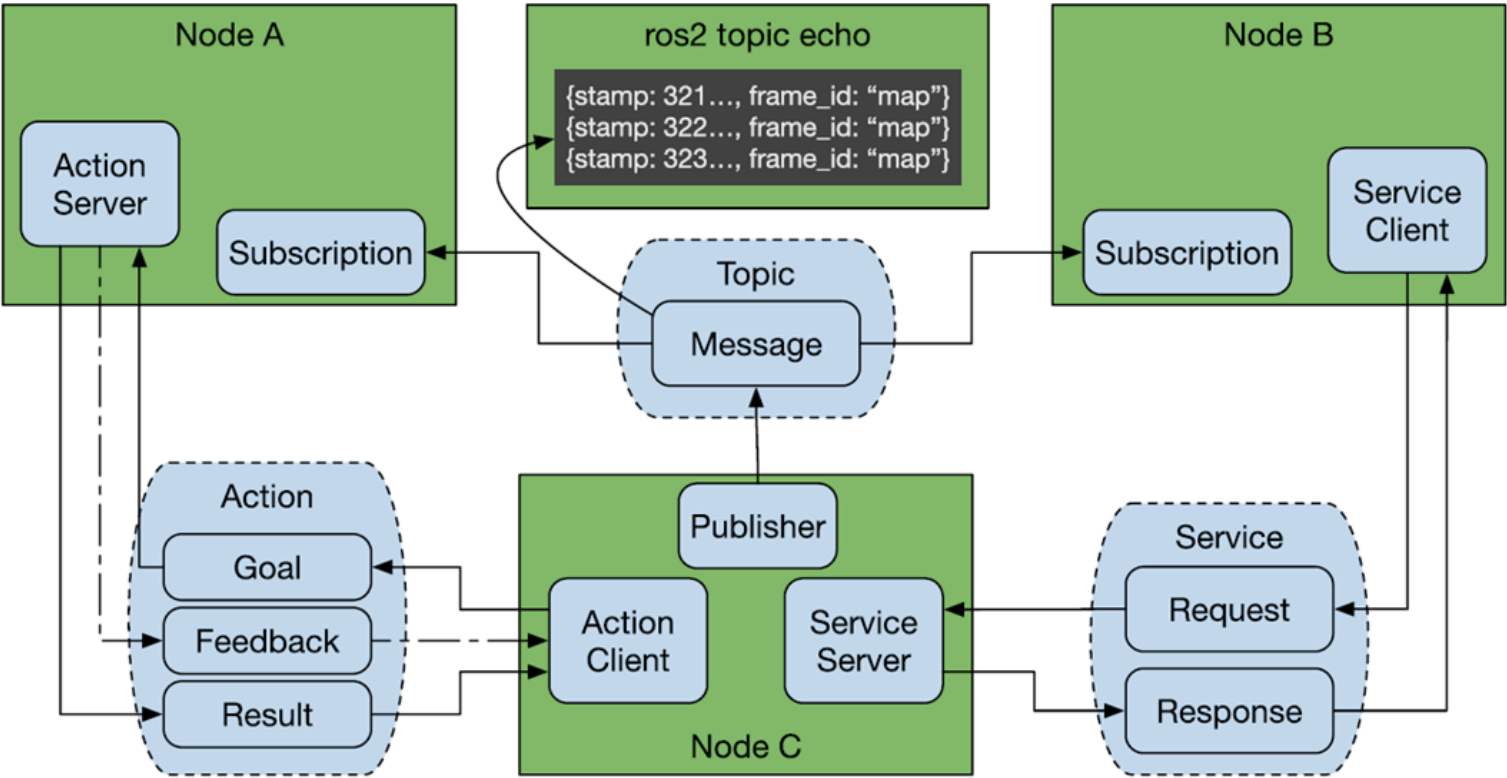
ROS 到底是什么？

Robot Operating System（机器人操作系统）。
它不是像 Windows/Linux 那样的传统操作系统。
一个运行在 Linux 之上的**元操作系统（中间件框架）。

- **三大核心能力：**
 - i. **硬件抽象**（屏蔽底层差异）
 - ii. **进程间通信**（节点对话）
 - iii. **软件包管理**（功能复用）

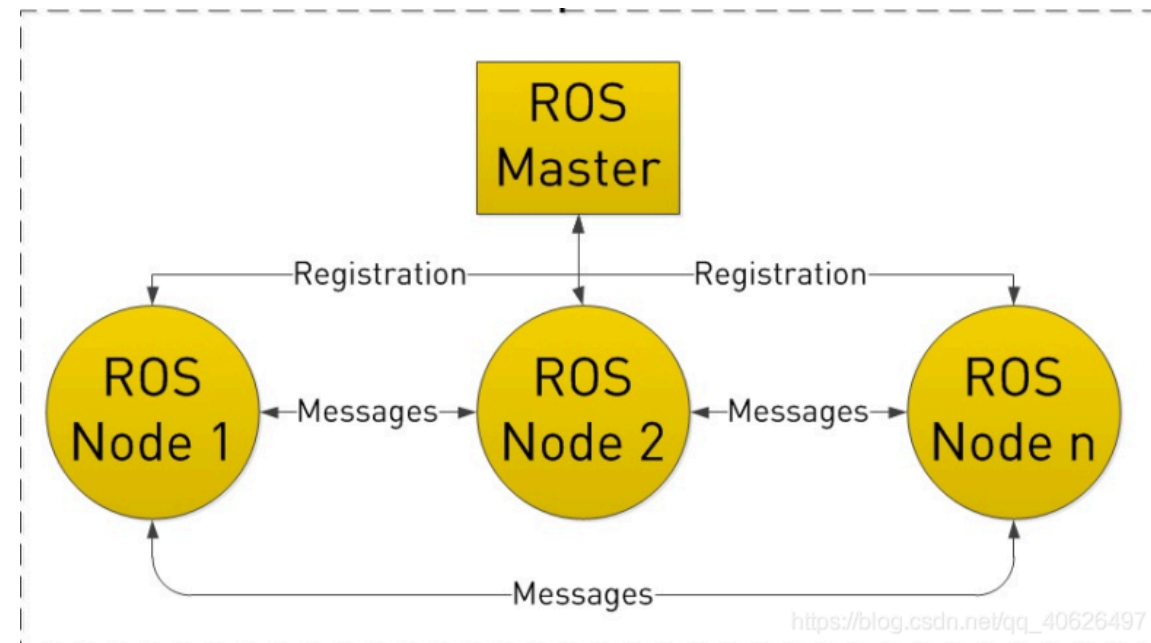
分别对应 “分布式、模块化、可复用”

ROS的框架



ROS 的核心架构：计算图

- **节点 (Node):** 执行具体任务的独立进程（如：摄像头驱动、避障算法）。
- **节点管理器 (Master):** 充当“通讯录”，帮助节点互相找到对方。（在 ROS2 中已经优化，改为分布式通信）
- **消息 (Message):** 节点之间传递的数据载体。

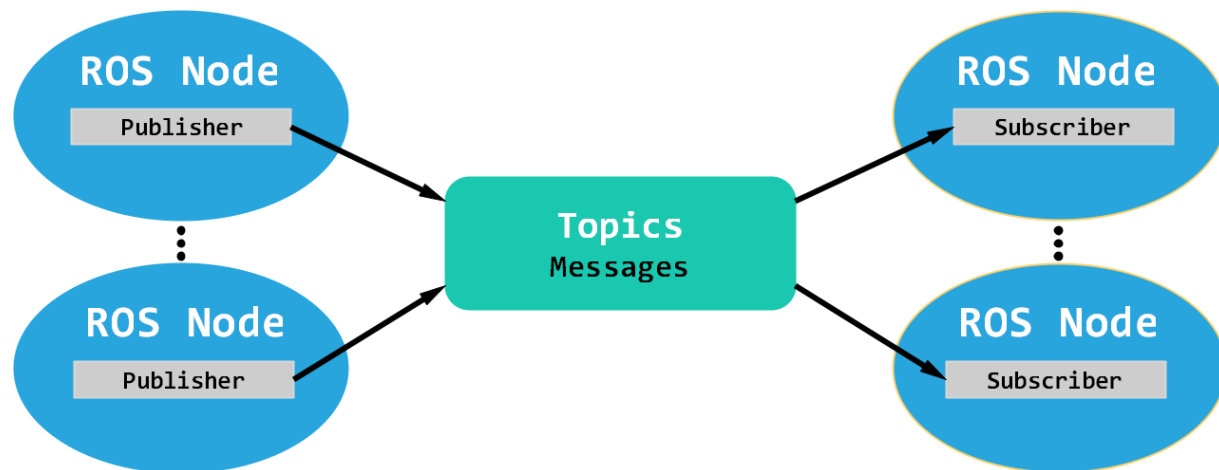


核心概念一：话题 (Topic)

适用场景： 连续的数据流（如传感器数据、控制指令）。

- **通信模型：** 发布/订阅 (Publish/Subscribe)
- **特点：** 多对多，异步通信，不保证实时送达。

 **理解关键：** 就像广播电台，发射塔只管发，收音机只管收，互不干扰。

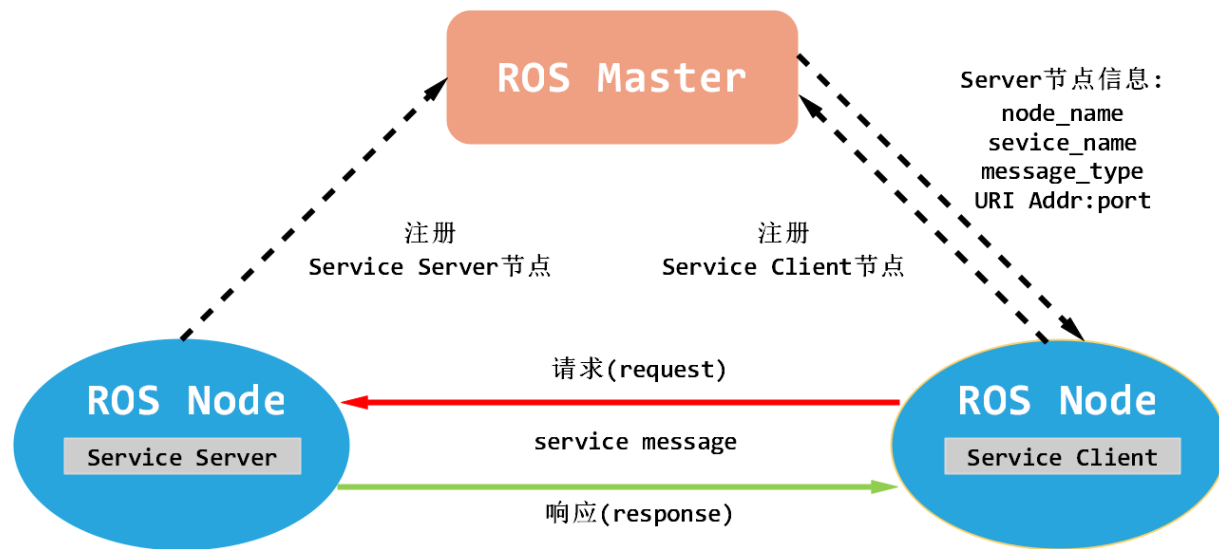


核心概念二：服务 (Service)

适用场景： 逻辑性的、偶尔调用的任务
(如：“拍照”、“开关灯”)。

- **通信模型：** 请求/响应 (Request/Response)
- **特点：** 同步通信，必须有反馈。

 **理解关键：** 就像打电话，你说一句，对方回一句，必须等到回应才算完成。

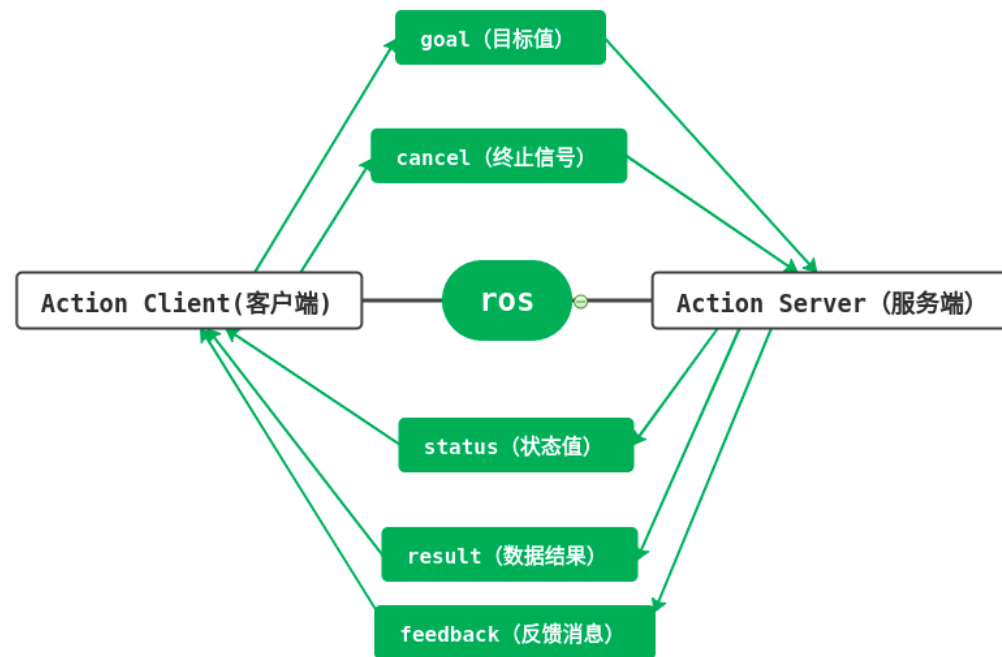


核心概念三：动作 (Action)

适用场景： 耗时较长、需要反馈进度的任务（如：导航到目标点）。

- **特点：** 结合了话题和服务的优点。
 - i. 有目标 (Goal) 。
 - ii. 有反馈 (Feedback, 比如 “还剩 5 米”) 。
 - iii. 有结果 (Result, 比如 “到达” 或 “失败”) 。
 - iv. 可取消 (Cancel) 。

 **理解关键：** 就像点外卖，下单是目标，骑手位置是反馈，送达是结果，随时可以退单。



CSDN @splendid.rain生

常用工具箱：让开发更简单

工具	作用	示意图关键词
rqt_graph	可视化节点关系图	连线图
rviz	3D 数据可视化	点云、机械臂模型
Gazebo	物理仿真环境	虚拟世界、碰撞检测
rosbag	数据录制与回放	录像机

案例演示：简易小车移动

任务： 让小车向前移动 1 秒。

1. **节点 A (键盘控制)：** 读取按键，向 `/cmd_vel` 话题发布速度消息。
2. **节点 B (底盘驱动)：** 订阅 `/cmd_vel`，解析速度值，控制电机转动。

 **实现逻辑：** 键盘节点不断发送前进指令，底盘节点收到指令后持续驱动电机，直到停止发送或收到停止指令。整个过程通过话题异步完成，无需等待对方响应。

https://www.bilibili.com/video/BV1R1wTeGEGd/?spm_id_from=333.337.search-card.all.click

学习路线建议

1. **基础准备：** Linux 常用命令 + Python/C++ 基础。
2. **核心概念：** 跑通 “小乌龟 (turtlesim)” 例程，理解话题与服务。
3. **进阶实践：**
 - 使用 URDF 搭建一个简单的机器人模型。
 - 在 Gazebo 中实现 SLAM 建图与导航。
4. **实战项目：** 参加 RoboMaster 或智能车竞赛。

总结

1. **ROS 是框架**：解决通信与复用问题。
2. **通信是核心**：掌握 Topic（流）、Service（逻辑）、Action（长任务）。
3. **工具是利器**：善用仿真与可视化工具提高效率。

Q & A

欢迎提问!